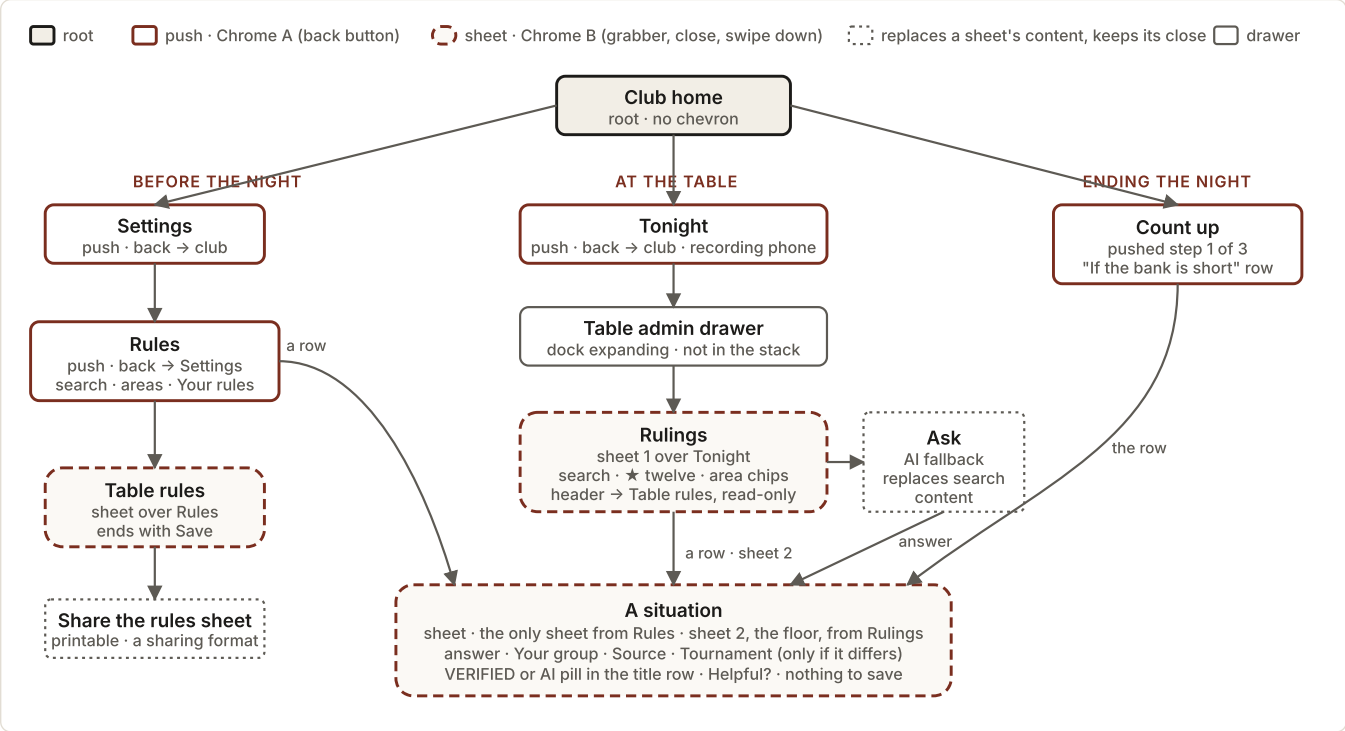
# Rules & rulings — a design

The Poker Club · a rules library for reference at the table, the group's own table rules, an AI fallback marked as such, and a review queue · revised 22 October 2026 · docs/rulings.md

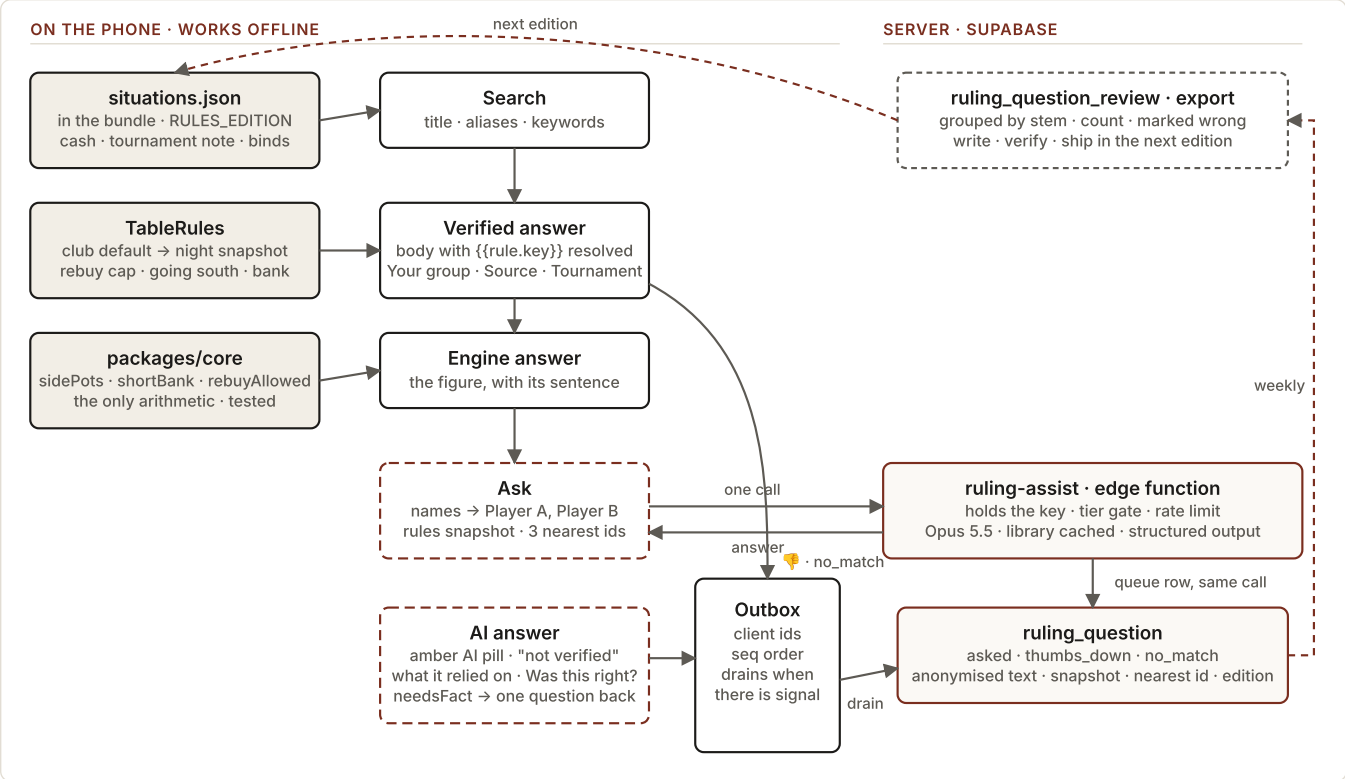
## Structure at a glance

Two pictures before the prose. The first is where the feature sits in the app and which chrome each screen wears. The second is how a question becomes an answer, and how an unanswered one becomes a verified answer in the next edition. Nothing is written to the night: the feature is reference only.

### Navigation




### Data and the loop




A host at a kitchen table, a pot in the middle, two people certain of opposite things. This is the feature that answers them from the phone that is already recording the night. It is a design, not a build: nothing in `apps/`, `packages/` or `supabase/` changes with this file. Written 20 October 2026 against `main` at `9a78e1c`, from `reports/Poker rules advice services.md` and the five note files behind it; revised 22 October after the owner's review (§ 1 items 7–8).

**How to read it.** § 1 is the decisions the owner has already made and this file builds on. § 2 is where the feature sits in the app and which chrome each screen wears. § 3 is how a host finds a situation in three taps. § 4 is the content model for a verified answer. § 5 is how the group's own rules feed the answers. § 6 is the AI fallback and how it is marked. § 7 is the review queue that turns unanswered questions into verified ones. § 8 is what is deliberately not recorded. § 9 is the first draft of the situation list. § 10 is copy, § 11 the checks, § 12 what is open, § 13 the build order.

Every string in this file that no board draws is marked **UNSURE** at the place it first appears, per the copy rule in `CLAUDE.md`. There are many, because no handoff has drawn any of this yet.

---

## 1 · The decisions this builds on

From the owner, 20 October, and from the research the week before:

1. **Inside the ledger app, not a separate app.** The research found no audience for standalone rules apps (0–22 ratings across the category), no willingness to pay for rules text, and a ledger that already holds the three things a ruling needs: the players, the stakes and the host.
2. **A default list of verified answers, written by us.** The app ships the list; the list is the product.
3. **AI for anything the list misses, visibly marked as AI.** The AI answer is never presented as ours.
4. **Every miss is captured for review**, so the list grows from what people actually ask rather than from what we imagine they ask.
5. **Cash game first.** Every answer is written for a cash game. Where a tournament ruling differs, the answer says so in a note that cannot be mistaken for the answer.
6. **Navigation is the hard part.** Seven areas, sixty situations, a host with one thumb and a table waiting.
7. **Reference only.** (22 October.) A ruling is read at the table and that is the end of it: nothing is written to the night, the log or the settled screen. The ledger stays money. What the app keeps is the *question*, for the review queue, never the *decision*.
8. **Verified and AI answers are both badged**, in the app's own pill: a check and VERIFIED on ours, AI in the amber the app already uses for anything waiting on somebody.

Two constraints from the research that shape the whole thing:

- **Robert's Rules of Poker cannot be bundled.** Its private-game edition bars commercial use without the late author's consent. Every answer here is written in our words, and cites Robert's Rules or the TDA rules as *source*, never quotes them at length.
- **General chatbots fail on exactly the procedural parts** — pot counting, straddles, the five-card rule (Silver, May 2025). So the AI never does arithmetic. Side pots, rebuy caps, rounding and the short bank are computed in `packages/core`, which is the only place arithmetic happens in this app and the only place it is tested. The AI explains; the engine counts.

---

## 2 · Where it lives

Three entry points, one library, two chromes. The pattern already exists in the app: **Money rules** is a push from Settings (the group's defaults, a place) and a sheet from Tonight (tonight's list, one thing to do). Rulings follow it exactly, which means nothing new for a host to learn about which gesture gets them back.

### 2.1 The screens

| Screen | Chrome | Opens from | Why that chrome |
|---|---|---|---|
| **Rules** — the library | **Push** (Chrome A), back → Settings | Settings, a new row under *Money rules* | A reference you browse and stay in. Club → Settings → Rules is two pushes below the root, the limit, so nothing pushes from it. |
| **A situation** — the answer | **Sheet** (Chrome B) | A row on Rules, a row on the Rulings sheet, the count-up hook, search | One thing: read one answer and swipe back to the table. From Tonight it must be a sheet in any case, because the Rulings sheet is already open and a sheet never pushes. |
| **Rulings** — at the table | **Sheet** over Tonight | The table-admin drawer, a new row | The host is mid-night and needs to get back to Tonight unchanged, by swipe. Same reason tonight's Money rules is a sheet. |
| **Table rules** — the group's parameters | **Sheet** over Rules (from Settings), or **read-only sheet** over Tonight | *Your rules* row; the Rulings sheet's header | Ends with Save → sheet. Over Tonight it is read-only, for the same reason `house-rules.tsx` is: a night's rules are a snapshot taken when it opened. |
| **Ask** — the AI fallback | Replaces the Rulings or Rules search content | *Nothing matches? Ask* | A step, not a place. |

Depth is legal everywhere: from Tonight, the Rulings sheet is the first sheet and a situation is the second, which `09-navigation.md` names as the floor, and nothing opens from a situation. From Settings → Rules, a situation is the only sheet.

### 2.2 The three entry points

**Settings.** A new Action row after *Money rules*: **Rules & rulings** (UNSURE — label). It pushes `/rules`. This is where a host reads before a night, sets the group's table rules, and shares the printable sheet.

**The table-admin drawer on Tonight.** A fifth row, **Rulings** (UNSURE — label; icon not drawn), between *Money rules* and *Pass the game*. It opens the Rulings sheet. The drawer is "the dock expanding in place" and does not join the navigation stack, so this costs nothing in depth. Only the phone recording the night sees the drawer; a watcher gets the same library from Settings.

**Count up.** When *In play* and *Accounted for* disagree, the discrepancy is already the headline. One row beneath the counted list — **If the bank is short** (UNSURE) — opens the short-bank situation directly, resolved against the group's rule. The count-up header is owned by the 6 September cut and this row must sit outside the block the cut draws. Whether it should exist at all is in § 12.

### 2.3 What is deliberately not an entry point

- **Not a dock button.** The dock has three slots and they are money.
- **Not a tab.** There is no tab bar.
- **Not the club home.** The root is the night and the book. A host who wants rules before a night goes to Settings, the same way they go for Money rules.
- **Not a sheet over a player card.** A ruling is about the table, not a player, and the player card already raises the amount sheet, which is the floor.

---

## 3 · Finding a situation in three taps

The research is specific about what a host needs at the table: the dozen judgement calls that start arguments (mucked winner, verbal versus chips, string bet, out of turn, exposed card) and the dozen money-procedure questions that rulebooks never cover (short bank, rebuy cap, going south, leaving). Lookups — who wins, kickers, blinds — are asked a hundred times more often but by beginners, usually before the night and usually of Google. So the two surfaces are weighted differently.

### 3.1 The Rulings sheet (at the table)

```
▬▬
Rulings                                  [×]
Cash game · your rules apply · edition 2026.1
┌──────────────────────────────────────────┐
│ 🔍  What happened?                       │
└──────────────────────────────────────────┘
AT THE TABLE
  Winning hand hit the muck                ›
  Said "raise", put in one chip            ›
  Bet in two motions                       ›
  Acted before their turn                  ›
  Card flashed while dealing               ›
  Someone has three cards                  ›
  Who shows first?                         ›
  Three people all in — the pots           ›
  Bank is short                            ›
  Wants to buy back in                     ›
  Leaving with a big stack                 ›
  Was that a slow roll?                    ›
MORE
  Betting · Showdown · Hands · Dealing · Money · Behaviour
Nothing matches?  Ask ›
```

Every string in the mock is UNSURE. The structure is the decision:

- **Search first**, because a host who knows what happened types two words. Search runs offline over title, aliases and keywords (§ 4), ranks by prefix match then substring, and shows at most six rows. A phrase that matches nothing shows the *Ask* row alone.
- **Twelve situations, by name of what happened**, not by rule. "Winning hand hit the muck", not "Dead hands". The host is describing, not classifying. These twelve are the research's ranking (§ 9 marks them ★) and the list is fixed per content edition, not personalised — a host should find the same row in the same place every week.
- **Areas as a chip row** underneath, for the long tail. Tapping an area replaces the twelve with that area's list; the sheet's search and close stay.
- **The sub-line names the edition and that the group's rules apply.** A ruling read from this sheet is dated by it.

### 3.2 The Rules push (from Settings)

The same library as a place. Search at the top; then the seven areas as sections, each a captioned list of its situations in the order § 9 gives; then **Your rules** (UNSURE) as the last section, with the parameters the group has set and the ones still at default, and a **Share the rules sheet** row (UNSURE) that produces the one-page printable the research found hosts search for ("poker rules printable", "poker cash game house rules").

### 3.3 The situation sheet

```
▬▬
Winning hand hit the muck   ✓ VERIFIED   [×]
Showdown · edition 2026.1

If the cards are face up on the table, they
play. Cards speak: the dealer reads them,
and a hand the player misread still wins.

If they went face down into the muck and
```

```
can be pulled out with no doubt which they
were, they are live. If there is any doubt,
the hand is dead and the pot goes to the
best hand still showing.

YOUR GROUP
  Who decides a dispute      Lena (host)
  Change ›

SOURCE
  Robert's Rules of Poker, Showdown · TDA 2026 r.15, 17

TOURNAMENT
  Different in a tournament ›

Helpful?                                  👍 👎
```

Decisions in the sheet:

- **The badge is in the title row**, where the app puts every state: the `Pill` component, plain tone, a check glyph and VERIFIED (UNSURE — label). Plain, not green: the token file is explicit that if something in this app is coloured, it is money. The AI answer's pill is the same component in the amber tone (§ 6.4), so the two states sit in the same place and differ in colour and word, and a host can tell them apart across the table.
- **The answer is the body, plain prose, at most three short paragraphs.** No bullets in an answer: the host reads it aloud.
- **Your group** is the house-rule binding (§ 5): the parameter this situation depends on, with its value, and *Change* which goes to Table rules. When the parameter is unset, the row reads the default and says **Not set for your group** (UNSURE) so a host knows the answer is ours, not theirs.
- **Source** is a citation, never a quote (§ 1).
- **Tournament** is a collapsed row, muted, and only present when the ruling differs. Expanded, it is one or two sentences in the same muted colour under a **TOURNAMENT** caption. The cash answer stays the only text at body weight on the screen. When the ruling is the same the row is absent — saying "same in a tournament" sixty times teaches people to stop reading the row.
- **No primary button.** The sheet ends at *Helpful?*; there is nothing to save (§ 8). A host reads it and swipes it away.
- **Helpful?** feeds the review queue (§ 7). A 👎 on a verified answer is the strongest signal we get.

---

## 4 · The content model

The library ships inside the app as data, versioned by edition, so it works at a kitchen table with no signal and so a ruling can say which edition it was read from. One file, `packages/core/src/rulings/situations.json` (path UNSURE — wherever the engine's own data lives), typed and linted by a test.

```
interface Situation {
  id: string;                  // 'showdown.mucked-winner' — stable for ever; rows are keyed by it
  area: Area;                  // 'betting' | 'showdown' | 'hands' | 'dealing' | 'money' | 'behaviour'
  atTheTable: boolean;         // one of the twelve on the Rulings sheet
  order: number;               // position within its area
  title: string;               // what happened, as a player says it
  aliases: string[];           // other ways people say it — search reads these
  keywords: string[];          // the words search matches on
  cash: Answer;                // the ruling, for a cash game — always present
  tournament?: {               // present only when the ruling differs
    note: string;              // one or two sentences
    source: Citation[];
  };
  binds?: TableRuleKey[];      // the group parameters the answer reads (§ 5)
  computes?: 'sidePots' | 'shortBank' | 'rebuyCap';   // hands the numbers to the engine
  related: string[];           // ids
  status: 'verified' | 'draft';
  verified?: { on: string; by: string };   // ISO date; a name, for the file, not a screen
}
```

```
interface Answer {
  body: string;                // ≤ 3 paragraphs; may contain {{rule.key}} placeholders
  source: Citation[];
}

interface Citation {
  work: 'roberts' | 'tda' | 'house';
  ref: string;                 // 'Showdown', 'r.15', or the TableRuleKey for 'house'
}
```

Rules of the file:

- **`id` never changes and is never reused.** A queued question (§ 7) points at it across editions.
- **`cash` is required; `tournament` is present only when it differs.** The lint test refuses a `tournament` whose note is empty and refuses a `cash` body over three paragraphs.
- **Every `{{rule.key}}` in a body must name a key in `binds`**, and every key in `binds` must exist in `TableRules` (§ 5). The test resolves every body against a fully-set and a fully-unset `TableRules` and asserts both render.
- **`status: 'draft'` ships but does not show.** It is how a situation gets into the file ahead of its answer being verified, so that search can capture interest in it (§ 7) without presenting it as ours.
- **The edition is one constant**, `RULES_EDITION = '2026.1'`, shown on the Rulings sheet's sub-line, on every situation sheet, and stamped into every queued question.

Why data in the bundle rather than fetched: the night is offline-first and a ruling is needed in the thirty seconds the argument lasts. An edition ships with an app release. Fetching a newer edition between releases is in § 12.

---

## 5 · The group's own rules

The research's third finding is that home games argue about money procedure and that no rulebook covers it: rebuy caps, going south, notice before leaving, who absorbs a short bank, rounding, IOUs. A 60-vote poll on the rebuy cap had no majority. So these cannot have one right answer; they are **parameters the group sets**, and the answer reads them.

### 5.1 `TableRules`, beside `MoneyRule`

`MoneyRule` is what comes *off* the table at settle-up — the bill, the piggy bank, a host fee — and `ruleText.ts` turns each one into a sentence. `TableRules` is how the table is *run*, and it is a different thing: one object per group, not a list, with no arithmetic of its own except where § 1 sends it to the engine.

```
interface TableRules {
  buyIn: { min: Money; max: Money } | null;       // the standard buy-in already in Settings is the default max
  shortBuy: 'one' | 'none' | 'any';               // Robert's: one short buy per session
  rebuyCap: { kind: 'none' } | { kind: 'largestStack'; percent: 50 | 75 | 100 } | { kind: 'fixed'; amount: Money } |
  goingSouth: 'forbidden' | 'allowed' | null;     // chips off the table mid-session
  leaveNotice: 'none' | 'oneOrbit' | 'thirtyMinutes' | 'oneHour' | null;
  returnWithin: 'oneHour' | 'never' | null;       // Robert's: back within an hour at the amount removed
  runItTwice: 'byAgreement' | 'never' | null;
  chops: 'allowed' | 'never' | null;
  straddle: 'none' | 'underTheGun' | 'button' | 'any' | null;
  button: 'dead' | 'moving' | null;
  rabbitHunt: 'allowed' | 'never' | null;
  checkRaise: 'allowed' | 'never' | null;
  shortBank: 'hostAbsorbs' | 'splitAll' | 'splitLosers' | 'decideThen' | null;
  credit: { kind: 'none' } | { kind: 'deadlineDays'; days: number } | null;
  decider: { kind: 'host' } | { kind: 'player'; playerId: PlayerId } | { kind: 'notInHand' } | null;
}
```

`null` means **not set**, and the answer says so. Rounding is **not** here: the night already carries a `RoundingMode`, and the rounding situation reads that.

### 5.2 The chain, and the snapshot

Table rules follow the money rules' chain exactly, because a host already understands it from `club-rules.tsx`'s own comment:

`this night (snapshot at open)  →  club default  →  not set`

- The club default is edited on the **Table rules** sheet from Settings → Rules.
- A night takes a **snapshot** when it opens, next to its `rules` snapshot. A ruling read on the night cites the snapshot, not the current default, so a rule changed on Tuesday does not rewrite what Saturday's ruling said.
- **No mid-night edit**, for the reason `house-rules.tsx` gives: changing a rule a night is running under is an act that needs a record, and until there is a record there is no edit. Over Tonight the Table rules sheet is read-only, with a line saying so (UNSURE — "Set before the night opens, in Settings").

### 5.3 How an answer reads them

A body says `{{rule.rebuyCap}}` and the renderer substitutes a sentence from a small map in core, the way `rateLabel` does for money rules:

| Key | Set | Not set |
|---|---|---|
| `rebuyCap` | *Your group caps a rebuy at half the biggest stack.* | *Your group has not set a cap. Most groups cap at half the biggest stack; some allow a full match.* |
| `shortBank` | *Your group's rule: the host covers a short bank.* | *Your group has not decided. The common rule is that whoever ran the bank covers it; some groups split it.* |
| `decider` | *Lena decides disputes tonight.* | *Nobody is named. Hosts usually decide; when the host is in the hand, pick someone who is not.* |

All UNSURE. The point is the shape: a set rule reads as the group's own, an unset one reads as ours and is honest about it, and the **Your group** row on the sheet gives the *Change* link either way.

### 5.4 Where the numbers go

Three situations compute rather than explain, and they go to `packages/core`:

- **`sidePots(stacks, bets)`** — several players all in for different amounts; who is eligible for which pot. Pure, tested against the handoff's canonical night and against the textbook three-way example.
- **`shortBank(rules, inPlay, accountedFor, players)`** — how much is missing and who carries it under the group's `shortBank` rule. Reads the same `balance` the count-up header reads; never its own sum.
- **`rebuyAllowed(rules, stacks, player, amount)`** — yes or no, and the cap.

The AI (§ 6) is never asked these. If a question routes to one of them, the answer is the engine's figure with the engine's sentence under it.

---

## 6 · The AI fallback

### 6.1 When it runs

Only when search finds no verified situation, or when the host taps *Ask* under a result that did not fit. It never runs on its own, never pre-empts a verified answer, and never runs for a situation whose `computes` is set — those go to the engine, and a question the engine can answer is routed by search to the situation, not to the model.

### 6.2 Where it runs

**On the server, never on the phone.** A Supabase Edge Function, `ruling-assist` (name UNSURE), holds the API key, checks the caller's membership tier, rate limits per group, and writes the queue row (§ 7) in the same call. The phone sends:

- the question as typed;
- the game kind, always `cash` today;
- the night's `TableRules` snapshot (or the club default when there is no night);

- the ids of the three nearest verified situations, by search score, so the model answers in the library's own terms and the response can say *closest verified situation: …*;
- the content edition.

Names are **not** sent as names. The phone replaces roster names in the question with *Player A*, *Player B* before sending — it knows the roster — and maps them back on display. The queue stores the anonymised text (§ 7).

### 6.3 The call

Claude Opus 5.5 ( `claude-opus-5-5` ), one request, structured output, adaptive thinking at `effort: 'medium'`. Not an agent: there is no tool to call and no loop to run. The whole verified library is the system prompt, under a `cache_control` breakpoint with a one-hour TTL, so the per-question cost is the question and the answer, not the library. The server-side `fallbacks: "default"` parameter is on, so a refusal on the primary model is retried on a fallback inside the same call; a refusal that survives that is shown as *Couldn't answer this one* (UNSURE) and still queued.

The system prompt says, in substance:

- You are answering for a **home cash game**. These are the verified situations; prefer their language. Cite the rule you rely on by name.
- **Do not do arithmetic.** If the question needs a pot, a stack or a share counted, say which calculator answers it and stop.
- **Do not invent a house rule.** If the answer depends on a parameter the group has not set, say the common default and that the group has not chosen.
- If the ruling would differ in a tournament, say so in one sentence, separately.
- If a fact is missing that changes the ruling (were the cards face up? had anyone acted?), ask for it instead of guessing.

The response shape, enforced by `output_config.format`:

```
interface AssistRuling {
  answer: string;                       // ≤ 3 paragraphs, the host's words
  reliesOn: Citation[];                 // what it cited
  tournamentDiffers: string | null;     // one sentence or null
  needsFact: string | null;             // a question back, or null
  nearestSituation: string | null;      // a verified id, or null
  confidence: 'high' | 'medium' | 'low';
}
```

`needsFact` renders as one question with a text field and a *That's it* button (UNSURE); the second call carries the first answer and the fact. Two rounds at most, then the answer stands as given.

### 6.4 How it is marked

The AI answer wears the situation sheet's layout with four differences, and all four are on the screen at once:

1. The title is the host's question, and the sub-line reads **AI answer · not verified · edition 2026.1** (UNSURE).
2. A **status pill** after the title — the same `Pill` the app uses for *Not settled* — reading **AI** (UNSURE), in the **amber tone with its 1px amber border**: the token the app reserves for *pending, unclaimed, unpaid*, which is exactly what an unverified answer is. Louder than the muted tone, and still not a colour the app spends on anything but waiting.
3. **Source** reads *What the AI relied on* (UNSURE) and lists `reliesOn`.
4. The footer is **Was this right?** (UNSURE) 👍 👎, and under it one line: *We'll check this one and add a verified answer* (UNSURE).

A verified answer carries none of these; it carries the check and VERIFIED instead. The pill is the thing a host can see across the table.

### 6.5 Cost, so the tier decision has a number

| | |
|---|---|
| Library in the system prompt | ≈ 60 situations × 250 words ≈ 20k tokens, cached |
| Per question, uncached | question + rules snapshot ≈ 600 tokens in, ≈ 400 out |
| Per question, at list price | under one cent with the cache warm; a few cents cold |

Which tiers get the AI is in § 12. The library itself should be free: the research names the printable rules sheet as the one acquisition hook in the category, and a library nobody can read captures no questions.

---

## 7 · The review queue

Every question that reaches *Ask*, every 👎, and every search that ended with no tap is a row we want. The loop is: capture → cluster → write → ship → the question stops arriving.

### 7.1 Capture

One row per event, written through the outbox like everything else the phone records, so a question asked with no signal arrives when the signal does:

```
create table ruling_question (
  id              uuid primary key,          -- client-generated: the idempotency key
  group_id        uuid not null references club(id),
  night_id        uuid references session(id),   -- null from Settings
  asked_at        timestamptz not null,
  kind            text not null check (kind in ('asked', 'thumbs_down', 'no_match')),
  question        text not null,             -- anonymised: Player A, Player B
  situation_id    text,                      -- the verified id a 👎 was on, or the nearest match
  answered_by     text not null check (answered_by in ('verified', 'ai', 'engine', 'none')),
  ai_answer       jsonb,                     -- the AssistRuling, when answered_by = 'ai'
  ai_model        text,                      -- the model id that answered
  rules_snapshot  jsonb not null,            -- the TableRules the answer was read against
  feedback        text check (feedback in ('helpful', 'wrong')),
  edition         text not null,
  app_version     text not null
);
```

Policies: the phone that wrote it may insert and may update `feedback` on its own rows; nobody on the client side may read another group's rows; reading everything is the service role, for us. The table names a group and, when there is one, a session, and it is **not** a ledger entry: `verify.ts` sums money, and a question is not money.

`no_match` rows are written locally on every search that ended without a tap and flushed in one batch when the sheet closes, so a host typing three variants of one phrase produces three rows with one `asked_at` minute, which is exactly the signal we want and easy to collapse.

### 7.2 Review

At first, no admin screen: a SQL view and an export.

```
create view ruling_question_review as
select lower(regexp_replace(question, '[^a-z ]', '', 'gi')) as stem,
       count(*)                        as asked,
       count(*) filter (where feedback = 'wrong') as marked_wrong,
       max(asked_at)                   as last_asked,
       mode() within group (order by situation_id) as nearest,
       array_agg(distinct edition)     as editions
from ruling_question
group by 1 order by asked desc;
```

`npm run rulings:export` (UNSURE — script name) pulls the view and the raw rows to a CSV. The workflow is: read the top of the view each week; for each stem with more than a handful of rows, write a situation, verify it, ship it in the next edition; the next release's `no_match` count for that stem goes to zero, which is the check that it worked. A 👎 on a verified answer is reviewed first, every time.

An admin page can come later and is not on the build order.

### 7.3 What we do not keep

The question as typed, with names. The phone anonymises before send (§ 6.2) and the row holds only that. A group name is not in the row either; `group_id` is. Nothing in the row identifies a person who is not the host, and the host is identified only as the writer of the night, which the server already knows.

## 8 · What is deliberately not recorded

The first draft of this file saved a ruling to the night: a `night_ruling` row beside `night_pass`, a *Record this ruling* step with *Decided by* and a note, a row in `/log`, a phrase in the settled night's meta, a line on the watcher's feed. **The owner took it out on 22 October: the feature is for reference, and the night's record stays what it is today.** Three reasons stand behind that, and they are worth keeping so the idea does not come back by accident:

- **The ledger is money.** Every row the night carries is summed or verified by `packages/core`. A sentence about a mucked hand is neither, and the first non-money row invites the second.
- **A recorded decision is a thing people argue with later.** The research's heated threads are all "was this right?" — a record with a name on it is a thing to re-litigate at cash-out, not a thing that ends the argument.
- **Nothing in the review loop needs it.** The queue (§ 7) keeps the question, the rules it was read against and whether the answer helped. That is what we need to write the next edition. Who decided what, we do not need and should not hold.

What remains of the idea is the `decider` parameter in Table rules (§ 5), which answers *who decides a dispute* inside the answers, and nothing that writes.

---

## 9 · The situation list — first draft

Sixty situations, seven areas, in the order they should appear. ★ marks the twelve on the Rulings sheet. **Every one-line answer is a DRAFT for the owner to verify** before it is written up and marked `verified`; none of it is final copy. *T* says whether a tournament ruling differs. *Binds* names the group rule the answer reads, if any.

### Betting

| # | What happened | Cash answer (draft) | T | Binds |
|---|---|---|---|---|
| ★ | Said "raise", put in one chip | Facing a bet, one big chip without saying "raise" is a call. Said "raise" first: it is a raise, and the amount is what they then put in. | same | — |
| ★ | Bet in two motions | A bet goes in one motion or with the amount said first. Two trips to the stack without a number is a call of the first trip. | same | — |
| | "I'll see your bet… and raise" | The words "call" or "see" came first and they bind. It is a call. | same | — |
| ★ | Acted before their turn | A check, call or fold out of turn stands if nobody before them changes the action. If somebody does, the early action is taken back and they may act again. | same | — |
| | Verbal "all in", chips still in the stack | Said in turn, it binds. The chips follow. | same | — |
| | All-in for less than a full raise | It does not reopen the betting for anyone who has already acted. They can call the extra, not re-raise. | same | — |
| | Minimum raise | At least the size of the last bet or raise in this round. | same | — |
| | Check-raise | Allowed unless your group has turned it off. | same | checkRaise |
| | Straddle | Whatever your group allows: none, under the gun, the button, or anyone. A straddle is a blind raise and acts last before the flop. | **differs** — no straddles | straddle |
| | Betting out of turn on purpose | Treat it as the out-of-turn rule, and tell them once. Twice is an angle (see Behaviour). | same | — |
| | Mis-called the pot size | Whoever counted it, the pot is what is in it. Nobody is bound by a wrong count. | same | — |

### Showdown

| # | What happened | Cash answer (draft) | T | Binds |
|---|---|---|---|---|
| ★ | Winning hand hit the muck | Face up on the table: it plays, cards speak. Face down into the muck and certain which it was: live. Any doubt: dead. | same | decider |
| | Dealer mucked a hand by mistake | Certain and retrievable: restored. Otherwise dead — and what happens to that player's money already in is for the group to decide beforehand. | same | decider |
| ★ | Who shows first? | The last person who bet or raised on the river. If nobody bet on the river, the first player left of the button. Anyone may table at once. | same | — |
| | Misread their hand ("I have two pair" with a flush) | The cards decide, not the words. The best five of seven. | same | — |
| | Showed one card, mucked the other | The hand is dead unless both are tabled face up. Showing one is not showing a hand. | same | — |
| | Can I see the hand they mucked? | In a cash game the caller may ask to see a called hand; it is bad form to ask for any other reason. The group can turn this off. | **differs** — only on suspicion of collusion, floor only | — |
| | Showed a card to one player during the hand | Show one, show all: at the end of the hand anyone at the table may ask to see it. | same | — |
| | Tied hands | The pot splits. An odd chip goes to the first player left of the button. | same | — |
| ★ | Three people all in — the pots | Each pot holds only what everybody in it could match. The calculator lays it out. | same | computes |
| | Board plays (all five on the board) | Everyone left in the hand splits, kickers in the hand do not count. | same | — |
| | Showing a hand after everyone folded | They may; nobody can make them. | same | rabbitHunt |

## Hands & ties

| # | What happened | Cash answer (draft) | T | Binds |
|---|---|---|---|---|
| | Which wins — rankings | The ranking, one screen, no prose. | same | — |
| | Both have a pair: the kicker | Highest card outside the pair, then the next, up to five cards. | same | — |
| | Two pair against two pair | Highest pair first, then the second pair, then the kicker. | same | — |
| | Flush against flush | Highest card, then the next, down through all five. Suits never rank. | same | — |
| | Is A-2-3-4-5 a straight? | Yes, the lowest one. K-A-2-3-4 is not. | same | — |
| | Full house against full house | The three of a kind decides, then the pair. | same | — |
| | Only the board makes a straight | The board plays; split unless a hole card improves it. | same | — |
| | Does a flush beat a straight? | Yes. | same | — |
| | Six cards to a straight | A hand is five cards. The best five count. | same | — |

## Dealing

| # | What happened | Cash answer (draft) | T | Binds |
|---|---|---|---|---|
| ★ | Card flashed while dealing | One hole card seen by the table: it becomes the burn and the player gets the next card after the deal. The first card to a player, or two cards: misdeal. | same | — |
| ★ | Someone has three cards | Before anyone acts: misdeal. After action: that hand is dead, the rest play on and bets stay. | same | — |
| | Card found face up in the deck | A boxed card is as if it were not there. Next card replaces it. | same | — |
| | Flop dealt before betting was done | The flop goes back, the deck is reshuffled without the burn, and a new flop is dealt after betting finishes. | same | — |
| | Turn dealt early | The early card is set aside, the river card becomes the turn, then the set-aside card is shuffled back for the river. | same | — |
| | Dealt from the wrong seat, button in the wrong place | Before action: fix it and redeal. After action: the hand stands and the button is corrected next hand. | same | button |
| | Dealt to an empty seat | The hand is dead and goes to the muck; blinds already posted stay. Fix the deal next hand. | same | — |
| | Forgot to burn | Discovered before the card is seen: burn now. After: the card stands. | same | — |
| | Ran out of cards | Reshuffle the burns and the muck, never the folded hands that were tabled; deal the rest. | same | — |
| | Who deals, and who shuffles | The group's way. The common one: dealer shuffles, player on the right cuts. | same | — |

## Money & the bank

| # | What happened | Cash answer (draft) | T | Binds |
|---|---|---|---|---|
| ★ | Wants to buy back in | A rebuy is any amount between the minimum and the group's cap, between hands, never during one. | — | rebuyCap, buyIn |
| | Buying in short | One short buy a session is the usual rule; after that a full buy-in. | — | shortBuy |
| ★ | Leaving with a big stack | Nobody has to stay. Say so an orbit ahead if that is the group's rule, and do not come back tonight with less than you left with. | — | leaveNotice, returnWithin |
| | Took chips off the table | Going south. The group's rule; the usual one is never. The chips go back. | — | goingSouth |
| ★ | Bank is short | The count is the count. Who covers it is the group's rule; the usual one is whoever ran the bank. | — | shortBank, computes |
| | Bank is over | The host holds it until the owner is found; after that it is the group's. | — | shortBank |
| | Rebuy paid by app, not seen | No chips until the payment shows. A rebuy said aloud is a rebuy recorded. | — | credit |
| | Wants to play on credit | The group's rule; the usual one is none. If allowed, a deadline, and the ledger says who owes. | — | credit |
| | Running it twice | Only in a cash game, only with everyone in the pot agreeing, only when all are all in. | **differs** — never | runItTwice |
| | Chopping the blinds | If the group allows it and both agree before the flop, the blinds come back. | **differs** — never | chops |
| | Change at cash-out | Tonight's rounding rule says what happens to the odd dollars. | — | (night rounding) |
| | Host fee, time charge, rake | What the group has set in Money rules. Note that a fee can be the thing that makes a private game unlawful where you are. | — | (money rules) |
| | Left without paying | Not a rules question. Settle before leaving is the rule; the ledger says who owes whom; the group decides who plays next week. | — | credit |

### Behaviour

| # | What happened | Cash answer (draft) | T | Binds |
|---|---|---|---|---|
| ★ | Was that a slow roll? | Taking your time to table the winner when you know it is the winner. No rule, no penalty — a word from the host. | **differs** — penalties exist | decider |
| | Talking about the hand with others in it | Not during the hand. Fold the talker's hand if the group has agreed that; otherwise a warning. | **differs** — penalty | — |
| | Rabbit hunting | The group's rule; the usual one is never. | **differs** — never | rabbitHunt |
| | Angle shooting | Doing a legal thing to mislead. The host decides, and the group's stand-in when the host is in the hand. | same | decider |
| | Phone at the table | Etiquette. No cards on camera, no phone in a hand with a bet in front of you. | same | — |
| | Exposing a card on purpose | Allowed heads-up unless the group forbids it; multiway, the hand is dead. | **differs** — penalty | — |
| | Who decides a dispute | The host; when the host is in the hand, the person the group named. | — | decider |

### Your rules

Not situations, but the eight parameters most groups argue about, shown as the group's sheet: buy-in range, rebuy cap, going south, leaving, run it twice, straddle, short bank, credit. The printable is this list and nothing else.

---

## 10 · Copy

Three rules for the words, beyond `CLAUDE.md`'s:

1. **A title is what happened, in the words a player uses.** "Winning hand hit the muck", not "Dead hand at showdown". Search reads aliases for the other phrasings.
2. **An answer is read aloud.** Short sentences, no bullets, the ruling first and the reason second. The research's heated threads are all "was this right?", and a sentence the host can say is what ends them.
3. **The AI is named as AI every time it speaks.** A pill, a sub-line, a changed footer. Never only one of them.

The strings this file invents, all UNSURE, in one place so a handoff can replace them: *Rules & rulings · Rulings · If the bank is short · Your rules · Share the rules sheet · Not set for your group · Table rules · Set before the night opens, in Settings · AI answer · not verified · AI · What the AI relied on · Was this right? · We'll check this one and add a verified answer · Couldn't answer this one · That's it · Verified · Nothing matches? Ask · AT THE TABLE · MORE · YOUR GROUP · SOURCE · TOURNAMENT · Helpful?* · and every title and answer in § 9.

---

## 11 · The checks

`CLAUDE.md`: a screen bug neither check can see is not finished being fixed. What goes red if this comes back wrong:

- **`npm run check`** — a content lint over `situations.json` (§ 4's rules: every `id` unique and never removed between editions, `cash` present, `tournament` only with a note, every placeholder bound and resolvable both set and unset, ≤ 3 paragraphs, `status` known); `sidePots`, `shortBank` and `rebuyAllowed` against the canonical night and the textbook cases; the resolver's set-and-unset sentences.
- **`npm run check:ui`** — `/rules` and the Rulings sheet added to `ui-audit.mjs`'s route list, driven across the six devices; the situation sheet opened on the longest verified answer with the Your-group row, Tournament row and footer all present, measuring that nothing is cut off; the AI-marked variant opened through a stubbed `ruling-assist`, asserting all four marks are on screen at once; the verified variant asserting the VERIFIED pill and no primary button.
- **`npm run db:verify`** — `0021_rulings.sql` (name UNSURE) applied to the throwaway Postgres; a group's rows invisible to another group; `ruling_question` writable by its writer and readable by nobody else; no new column on any money table.

---

## 12 · Open

1. **Which tiers get the AI.** The library free for all is argued above. The AI on Regular and Full, or Full only, is the owner's call and lands in `membership.ts`, the one seam, as a fifth function.
2. **The count-up hook.** The short-bank row under the counted list is the one place this design touches a screen a cut owns. If the cut's author would not draw it, it goes, and the host reaches the situation from the drawer instead.
3. **Fetching an edition between releases.** Data in the bundle is simplest and always works offline. A fetched edition would let a verified answer ship the week it is written. Worth doing once the queue shows questions arriving faster than releases.
4. **Robert's Rules citation style.** Citing by section name is safe; quoting is not. Whether to cite at all, given the licence, is for the owner.
5. **The Table rules sheet over Tonight is read-only.** A group that discovers mid-night it never set a rebuy cap will want to set it then. The honest path is the one `house-rules.tsx` already takes — show it, do not edit it — and a mid-night change is a later feature with its own record.
6. **Dealer-mucked hand and the money.** § 9 leaves what happens to the mucked player's money as a group decision because the sources disagree. The owner may want a default.
7. **Two-player and short-handed blinds, dealer's choice games.** Out of the first edition on purpose; the research shows them as lookups, not disputes.

---

## 13 · Build order

Three steps, each shippable, each on its own branch and its own files per `CLAUDE.md`'s rule about parallel sessions. Step 1 touches Settings and nothing on Tonight; step 2 touches Tonight and the ending flow and nothing in Settings; step 3 is server-side and the two sheets' *Ask* content.

**Step 1 — the library and the group's rules.** `TableRules` in core with its resolver and the three calculators; `situations.json` with the ★ twelve and the Money area verified first, the rest `draft`; the content lint; the night snapshot; `/rules` as a push from Settings; the situation sheet; offline search; the Table rules sheet; the printable sheet as a sharing format. No server change. This step is useful on its own and is the acquisition hook.

**Step 2 — at the table.** `0021_rulings.sql` with `ruling_question`; the outbox op; the drawer row and the Rulings sheet over Tonight; the count-up hook if § 12 allows it; `no_match` and 👎 capture; the export script. Still no AI. After this step the queue is filling.

**Step 3 — the AI.** The edge function, the anonymiser, the *Ask* content, the four marks, `needsFact`, the tier gate in `membership.ts`. After this step every question gets an answer, and every answer we did not write says so.

Nothing here starts until the owner says so.